FINAL PROJECT REPORT

SMART GRID SYNC: DECOUPLED AI ENERGY OPTIMIZATION FRAMEWORK

This report delivers the final technical verification and architectural breakdown of the Smart Grid Sync platform. Built using a decoupled client-server interface, the framework implements real-time grid dynamics, localized Indian Time-of-Day (ToD) utility pricing, slab billing metrics, price elasticity, statistical anomaly detection, and NumPy-based ML predictors.

Project Access Links:

GitHub Repository: https://github.com/shikhasrivastava0574-afk/Smart-Grid-Sync

Live Demo: https://shikhasrivastava0574-afk.github.io/Smart-Grid-Sync/frontend/

Project Milestones Completed:

- Grid Frequency Localized to Indian IEGC Standard (50.00 Hz)
- Integrated Time-of-Day (ToD) Tariffs & Slab-Based Bill Calculator
- Programmed Price-Elastic Demand Response & Load Contractions
- Built Statistical Anomaly Detection & Glowing SVG Marker Tooltips
- Deployed /api/grid/trends Endpoint with 24-Hour DB Aggregations
- Formulated Personalized Recommendations log showing Dynamic Rupee savings

1. Final Week Achievements & Localizations

During this iteration, the framework was systematically migrated from basic generic values to a highly realistic smart grid decision-support platform designed around standard Indian regulatory guidelines. The core accomplishments include:

A. Indian Grid Frequency Alignment

Changed grid base frequency parameters from the US 60 Hz standard to the Indian standard of 50.00 Hz (regulated by the Indian Electricity Grid Code, IEGC). We limited normal frequency bounds to fluctuate safely between 49.10 Hz and 50.80 Hz. Database seeders generate telemetry logs centered on this 50 Hz scale.

B. Time-of-Day (ToD) Tariff & Slab-Based Bill Estimators

Implemented standard time-based surcharges and rebates. Slabs charge Rs. 4.50/unit for 0-100 units, Rs. 8.50/unit for 101-300 units, Rs. 12.00/unit for 301-500 units, and Rs. 15.00/unit above 500 units. A projected monthly slab bill is dynamically calculated and displayed in Streamlit dashboards.

C. Automated Demand Response & Consumer Elasticity

Integrated active load feedback loops where consumption contracts by 15% during critical peaks (> Rs. 12.00/kWh) and contracts by 8% during normal peaks (> Rs. 9.00/kWh). Off-peak slots (< Rs. 5.00/kWh) trigger a 5% load expansion, modeling automated consumer shaving.

2. Advanced Grid Analytics Implementation

Three advanced analytical systems were integrated into the core full-stack platform to provide better data visibility, warning notifications, and economic advice to operators:

A. Statistical Anomaly & Theft Detection

Telemetry is scanned in real-time. If grid frequency drifts past safety parameters (< 49.85 Hz or > 50.15 Hz), demand load deviates drastically from daily averages, or suspicious bypass theft is detected (sudden drop in load during peak pricing), an active anomaly is flagged. Blinking red pulsing markers are drawn dynamically on the frontend SVG chart lines with diagnostic hover tooltips.

B. 24-Hour Historical Trend Insights (/api/grid/trends)

Aggregates SQLite historical logs to present daily metrics directly on the dashboard panels: Peak Demand Time (e.g. 8:00 PM), Average vs. Peak Load ratios, Grid Stability Factor (evaluating standard deviation of frequency deviations), and detailed anomaly totals (including theft and frequency counts) over the last 24 hours.

C. Personalized Savings Recommendations Engine

Calculates specific load-shedding and battery arbitrage rewards in local Rupee values in real-time. For example, when rates spike, it displays the specific Rupees/hour saved if the manager shaves 15% of active demand. It also displays the hourly savings achieved by battery peak-shaving compared to grid energy.

D. Cryptographic HMAC Security Audit Ledger

Each smart-meter telemetry payload is cryptographically signed using HMAC-SHA256 with a unique pre-shared secret key. The SLDC backend verifies signatures upon packet arrival. If telemetry is spoofed or modified in transit, the signature check fails, triggering an immediate security anomaly alert and warning log on the control room dashboard.

3. System Architecture & API Specifications

Method	Endpoint Route	Payload & Logic Details
GET	/api/grid/status	Returns live status dict (load, frequency, battery, dynamic price, active anomaly type, status tex
GET	/api/grid/history	Retrieves last 144 logged grid metrics (24h) from SQLite DB including stored anomaly tags.
GET	/api/grid/forecast	Queries pure NumPy predictors to yield 24-step forecast arrays for demand load and solar.
GET	/api/grid/trends	Queries SQLite DB metrics, calculating daily peak, avg, stability, and anomaly totals.
POST	/api/grid/control	Updates temp, clouds, wind, and battery modes. Payload: { temperature, cloud_cover, battery_
POST	/api/grid/scenario	Sets grid scenario presets. Payload: { scenario: 'heatwave' | 'congestion' | 'storm' | 'cloudy' }.

Verification Summary

1. Deployed Backend Codebase: Tested and operating on Render Cloud Services.
2. Deployed Frontend Dashboard: Successfully served via GitHub Pages in secure HTTPS protocol.
3. Local SQLite seeding: Confirmed database creation and telemetry records populated without errors.
4. Advanced analytics endpoints checked: Confirming trends and anomaly checks execute in under 15ms.

4. Deployment Links & Interface Visuals

Project Code & Live Demo Links

- **GitHub Repository:** https://github.com/shikhasrivastava0574-afk/Smart-Grid-Sync
- **Live Interactive Demo:** https://shikhasrivastava0574-afk.github.io/Smart-Grid-Sync/frontend/

The smart grid dashboard is built using vanilla JS and CSS, applying glassmorphic design systems, interactive real-time SVG charting, and dynamic alerts. Screenshots from the live interface follow below.

Live UI Dashboards